

TECHNOLOGY



Designing Microsoft Azure Infrastructure Solutions: AZ:305

Designing an Application Architecture



Learning Objectives

By the end of this lesson, you will be able to:

- List the features of Azure Functions for efficient serverless computing
- Identify the three Azure messaging services that assist with delivering messages or events throughout a solution
- Outline the key components of Azure Queue Storage for efficient message queuing
- List the ways Azure Resource Manager Templates can be deployed for infrastructure as code



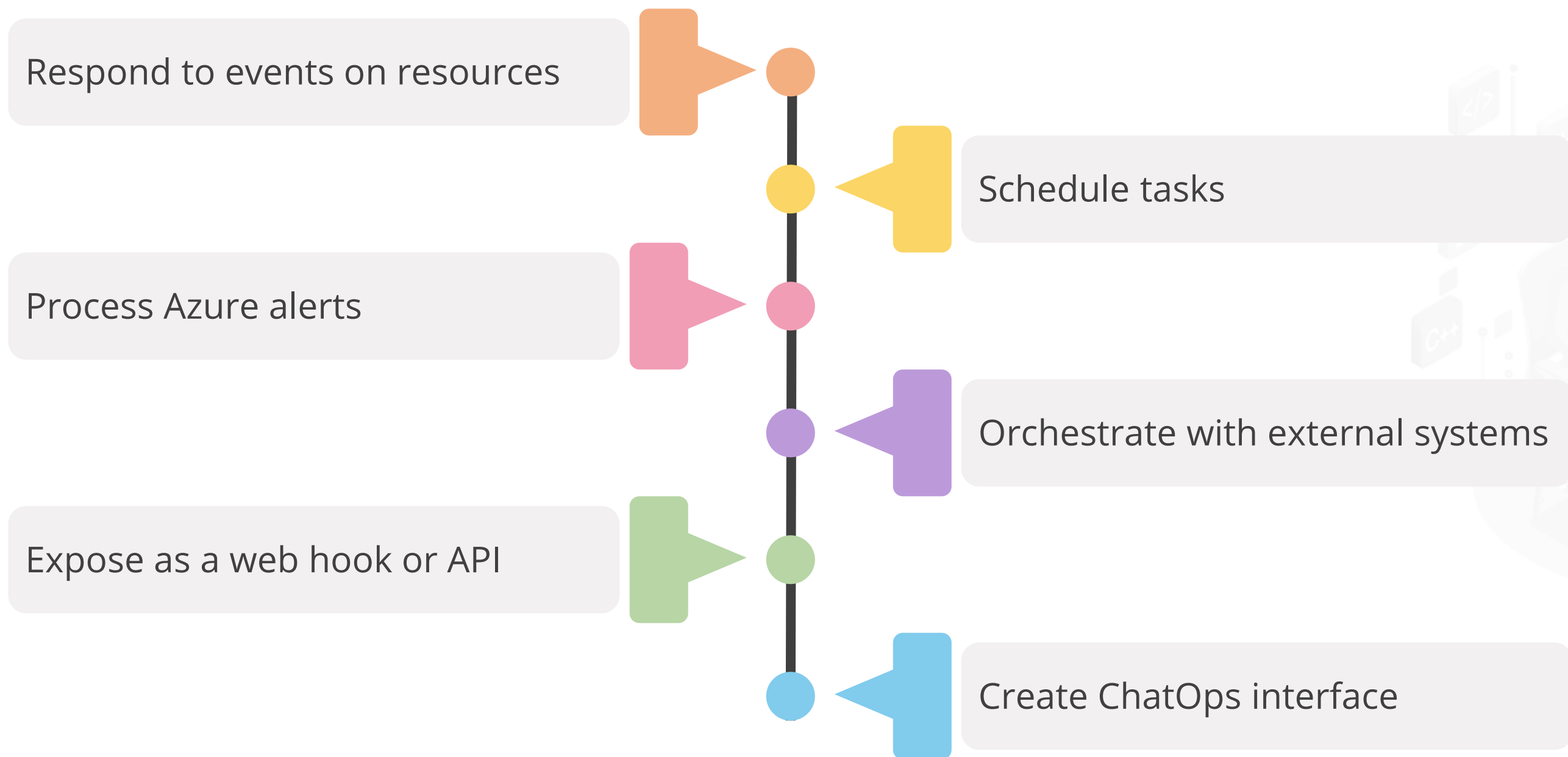
Recommend a Microservices Architecture

Serverless vs. Provisioned Architecture

Aspect	Serverless architecture	Provisioned architecture
Overview	Cloud provider manages infrastructure, scaling, and resource allocation; focus on code deployment.	Users manually allocate and manage resources, including scaling, maintenance, and monitoring.
Use cases	Ideal for event-driven workloads, microservices, APIs, real-time processing, and unpredictable scaling needs.	Suitable for predictable, long-running workloads, legacy applications, or when full control is required.
Benefits	Automatic scaling, reduced operational overhead, cost-effective with pay-per-execution pricing.	Greater control over infrastructure, customization, and suitable for stateful or legacy applications.

Event-Based Cloud Automation

Event-based automation scenarios work best with Azure Functions. They follow these common patterns:



Event-Based Cloud Automation: Architecture

The architecture consists of the following components:

Azure Functions

- Provides event-driven and serverless compute capabilities in the architecture
- Performs automation tasks when triggered by events or alerts

Azure Logic Apps

- Performs simple tasks that are easily implemented using the built-in connectors
- Allows users to perform tasks ranging from email notifications to integration with external applications

Event-Based Cloud Automation: Architecture

Event Grid

Offers built-in support for events from Azure services and custom events (also called **custom topics**)

Azure Monitor

Monitors critical conditions and takes corrective action using Azure Monitor action groups

Automation Action

Represents other services that a function can interact with to provide automation functionality

Azure Functions

It is a serverless solution that allows users to write less code, maintain less infrastructure, and save on costs.



It offers the latest resources required to maintain the functionality of the applications.

Azure Functions: Features

Supports a range of programming languages

C#, F#, Node.js, Python, PHP, batch, bash, or any executable

Supports custom dependencies

NuGet and NPM-based libraries

Integrates with the most popular OAuth providers

Facebook, Google, Twitter, and Microsoft account

Facilitates ease of code reuse

Reusable functions for developers

Azure Functions: Features

Employs a pay-per-use pricing model

Metered services

Integrates with other apps

Azure services and SaaS apps

Allows flexible development

Directly from the Azure portal

Provides an open-source option

Available on GitHub

Azure Functions: Use cases

Run code based on HTTP requests and schedule code to run at predefined times

Respond to high volumes of Azure Event Hubs events and handle Azure Service Bus queue and topic messages

Respond to Azure Event Grid events by using subscriptions and filters

Process new and modified Azure Cosmos DB documents, Azure Storage Blobs, and Azure Queue storage messages



Azure Logic Apps

It is a cloud platform where one can create and run automated workflows with little to no code.



Logic App simplifies how users design and build scalable solutions:

- The workflow starts with a trigger that fires when an event happens or when new data meets specific criteria.
- When the trigger fires, the Logic App's engine creates a logic app instance that runs the actions in the workflow.

Resiliency

Azure Functions

Handle HTTP timeouts:

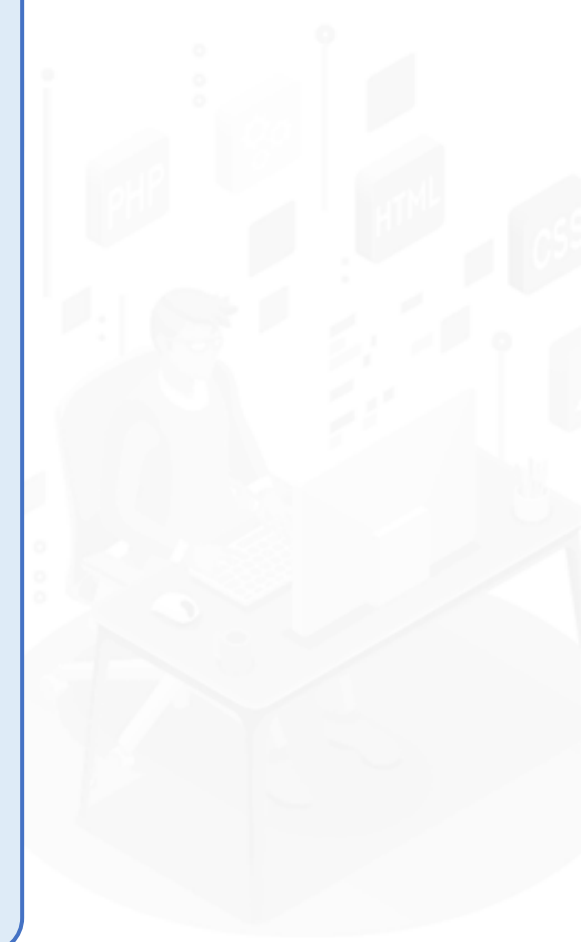
To avoid HTTP timeouts for a longer automation task, queue the event in a **Service Bus** and handle the actual automation in another function.

Log failures:

The function should log any failures in carrying out automation tasks. This allows for proper troubleshooting error conditions.

Concurrency:

The concurrency requirement for the automation function should be verified.



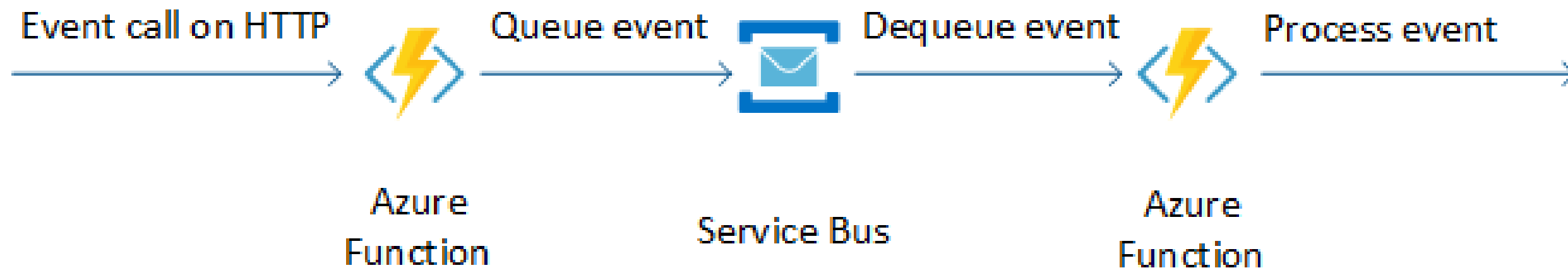
Resiliency

Event Grid

It checks for a high volume of generated events to determine if there are enough to clog the grid. The cost center workflow does not implement additional checks for this, as it only watches for resource creation events in a resource group.

Azure Monitor

If many alerts are generated and the automation updates Azure resources, throttling limits of the Azure Resource Manager might be reached. Avoid this situation by limiting the frequency of alert generation by Azure Monitor.



Security

It provides assurances against deliberate attacks and the abuse of valuable data and systems.

Control access to the function

Restrict access to an HTTP-triggered function by setting the authorization level. With anonymous authentication, the function is accessible with a URL:

`http://<APP_NAME>.azurewebsites.net/api/<FUNCTION_NAME>`

Control function access

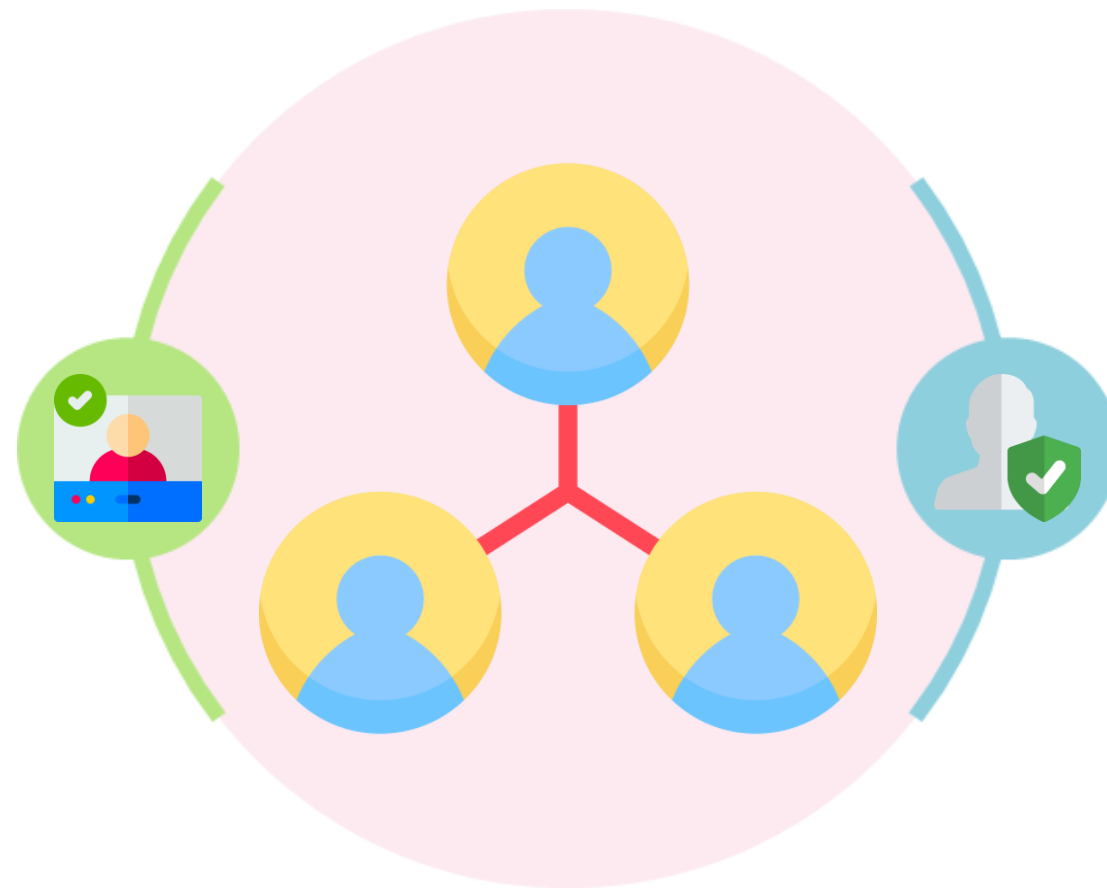
A Microsoft Entra feature simplifies how a function authenticates and accesses other Azure resources and services. The code does not need to manage the authentication credentials because it is managed by Microsoft Entra ID.

Security

The two types of managed identities are as follows:

System-assigned managed identities:

These are created as part of the Azure resource and cannot be shared among multiple resources. They are deleted when the resource is deleted.

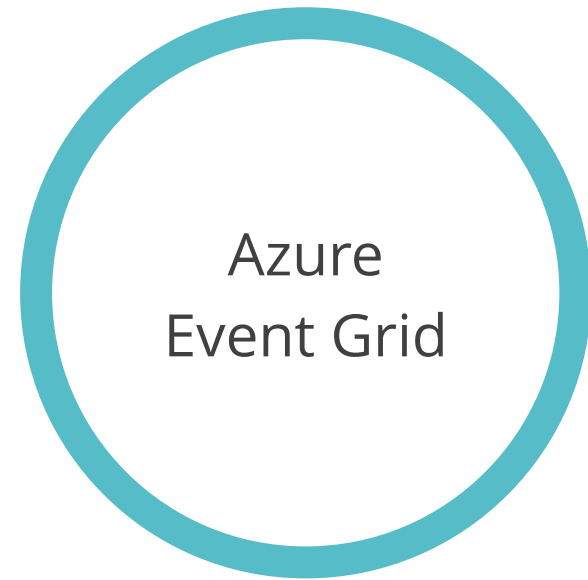


User-assigned managed identities:

These are created as stand-alone Azure resources. They can be shared across multiple resources and need to be explicitly deleted.

Azure Messaging Services

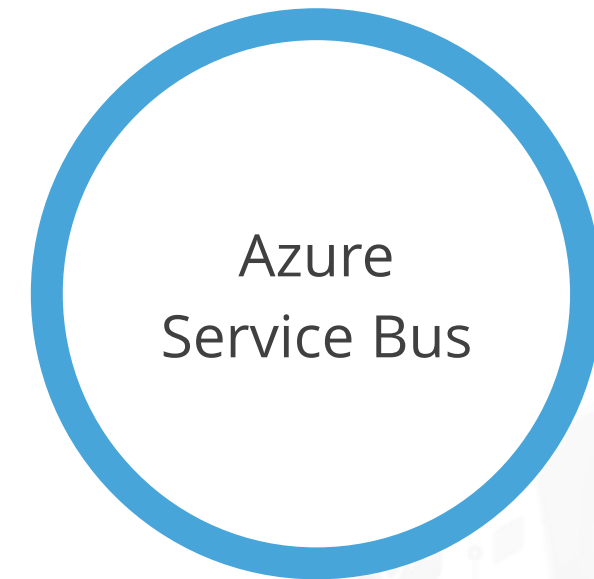
Azure offers three services that assist with delivering messages or events throughout a solution:



It is an eventing backplane that enables event-driven, reactive programming.



It is a big data streaming platform and event ingestion service.



It enables cloud-native applications to provide reliable state transition.

Event vs. Message

Event

- It is a lightweight communication mechanism in a software system that involves both a publisher and a subscriber.
- It is commonly used for broadcasts and is often considered ephemeral, implying that the communication may not be processed by any receiver if none is actively subscribing.

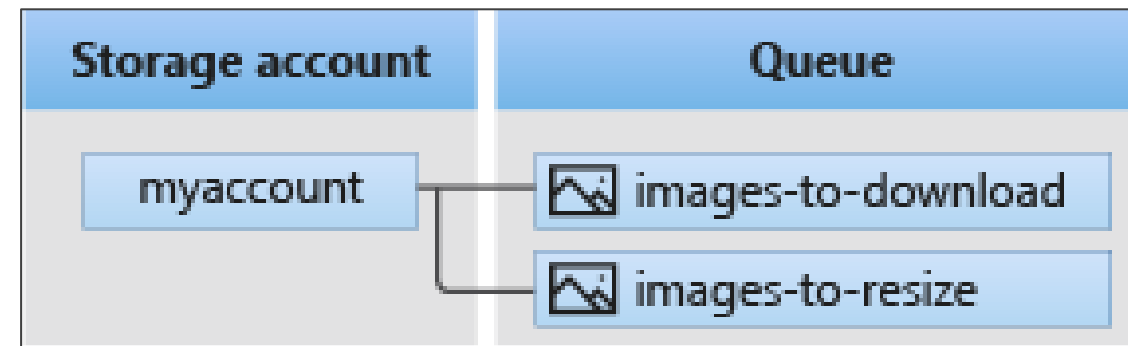
Message

- It contains raw data produced by one component, intending to be consumed by another component.
- It is employed in distributed applications when there is a need for assurance that the communication will be processed.



Azure Queue Storage

It is a scalable cloud messaging service for building highly decoupled and scalable cloud applications.



A queue message can be up to 64 KB in size.



Azure Queue Storage: Key Components

Storage account

- All access to Azure Storage is done through a storage account.
- It is the centralized management of storage resources.

Queue

- A queue contains a set of messages.
- The queue name must be all lowercase.

Message

- The maximum time-to-live can be any positive number or -1.
- If this parameter is omitted, the default time-to-live is a week.

Service Bus Queue vs. Service Bus Topic

Feature	Service Bus queue	Service Bus topic
Communication pattern	Point-to-point messaging	Publish/subscribe messaging
Use case	One-to-one communication and Point-to-point messaging	One-to-many communication
Message handling	Consume each message with one receiver	Consume each message with multiple subscribers
Receiver	Deliver each message to one customer	Multiple subscribers: each subscriber gets a copy of every message
Sender	Send a message to one queue using one sender	Direct one sender to dispatch messages to one topic
Message retention	Retained until consumed or expires based on TTL (Time-To-Live)	Retained until consumed or expires, similar to queues
Subscription filter	No subscription filter	Filter messages based on subscription rules

What is Kubernetes?

It manages container-based applications and their associated networking and storage components.



One can build and run modern, portable, microservices-based applications using Kubernetes to orchestrate and manage the availability of the application components.



Azure Kubernetes Service (AKS)

It simplifies deploying a managed Kubernetes cluster in Azure by offloading the operational overhead to Azure.



AKS provides a managed Kubernetes service that reduces the complexity of deployment and core management tasks, like upgrade coordination. The Azure platform manages the AKS control plane.

AKS: Features

- AKS supports Kubernetes RBAC.
 - It controls access and permissions to Kubernetes resources and namespaces.
- One can configure an AKS cluster to integrate with Microsoft Entra ID.
 - With Microsoft Entra integration, one can set up Kubernetes access based on existing identity and group membership.
- AKS nodes run on Azure Virtual Machines (VMs).
 - With AKS nodes, one can connect storage to nodes and pods, upgrade cluster components, and use GPUs.
- AKS offers a multiple Kubernetes versions.
 - As new versions become available in AKS, one can upgrade the cluster using the Azure portal, Azure CLI, or Azure PowerShell.



Evaluating Azure Kubernetes Service (AKS)



Duration: 10 Min.

Problem Statement:

You have been given a project to evaluate Azure Kubernetes Service (AKS) for running multitier applications and test its deployment and scaling capabilities to simplify management overhead.

Outcome:

The evaluation demonstrates that AKS can significantly enhance operational efficiency and reliability, making it a strong choice for managing multitier applications in the cloud.

Note: Refer to the demo document for detailed steps:
01_Evaluating_Azure_Kubernetes_Service

ASSISTED PRACTICE

Assisted Practice: Guidelines

Steps to be followed:

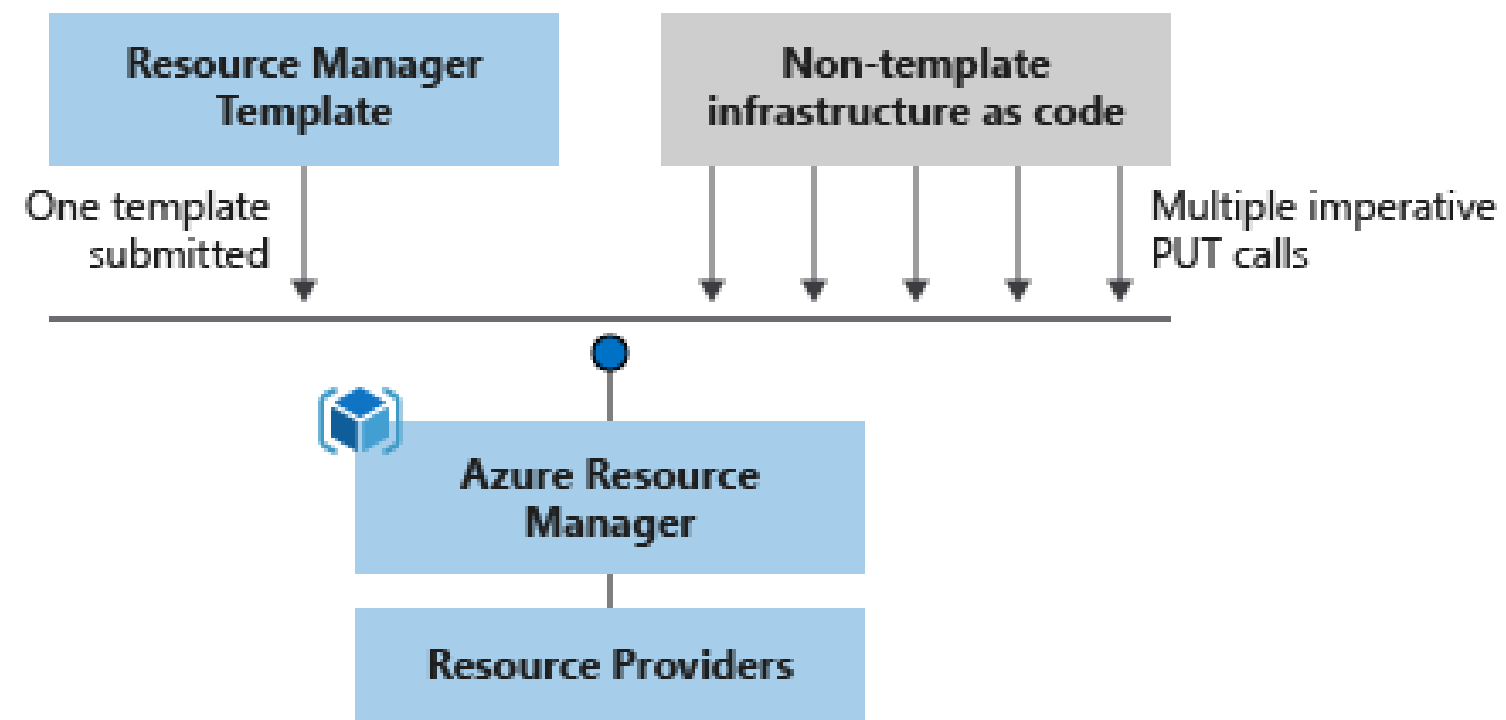
1. Register the Microsoft Kubernetes resource providers
2. Deploy an Azure Kubernetes Service (AKS) cluster
3. Deploy pods into the Azure Kubernetes Service (AKS) cluster
4. Scale containerized workloads in the Azure Kubernetes Service (AKS) cluster



Recommend an Orchestration Solution for Deployment and Maintenance of Applications

Azure Resource Manager (ARM) Templates

They are used for implementing infrastructure as code for Azure solutions.



The template is a JavaScript Object Notation (JSON) file that defines the infrastructure and configuration for the project.

ARM Templates: Advantages

Declarative syntax

ARM templates allow one to create and deploy an entire Azure infrastructure declaratively.

Repeatable results

Templates are idempotent, which means one can deploy the same template many times and get the same resource types in the same state.

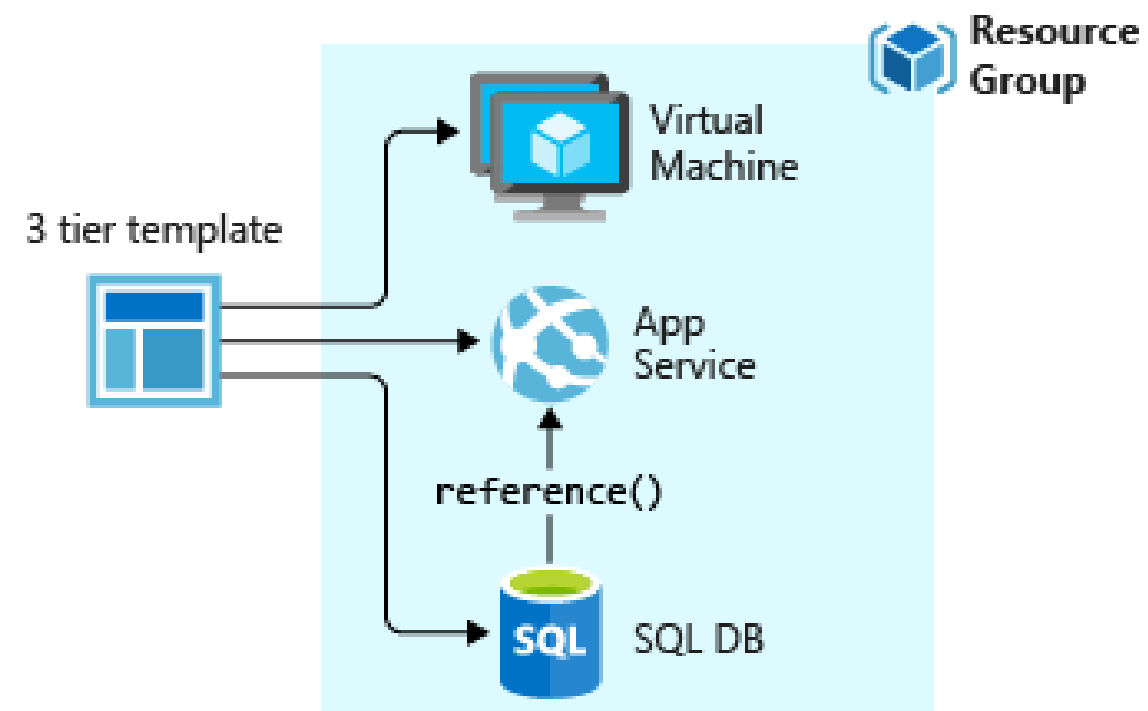
Orchestration

Resource Manager orchestrates the deployment of interdependent resources, so they're created in the correct order.



ARM Template: Design

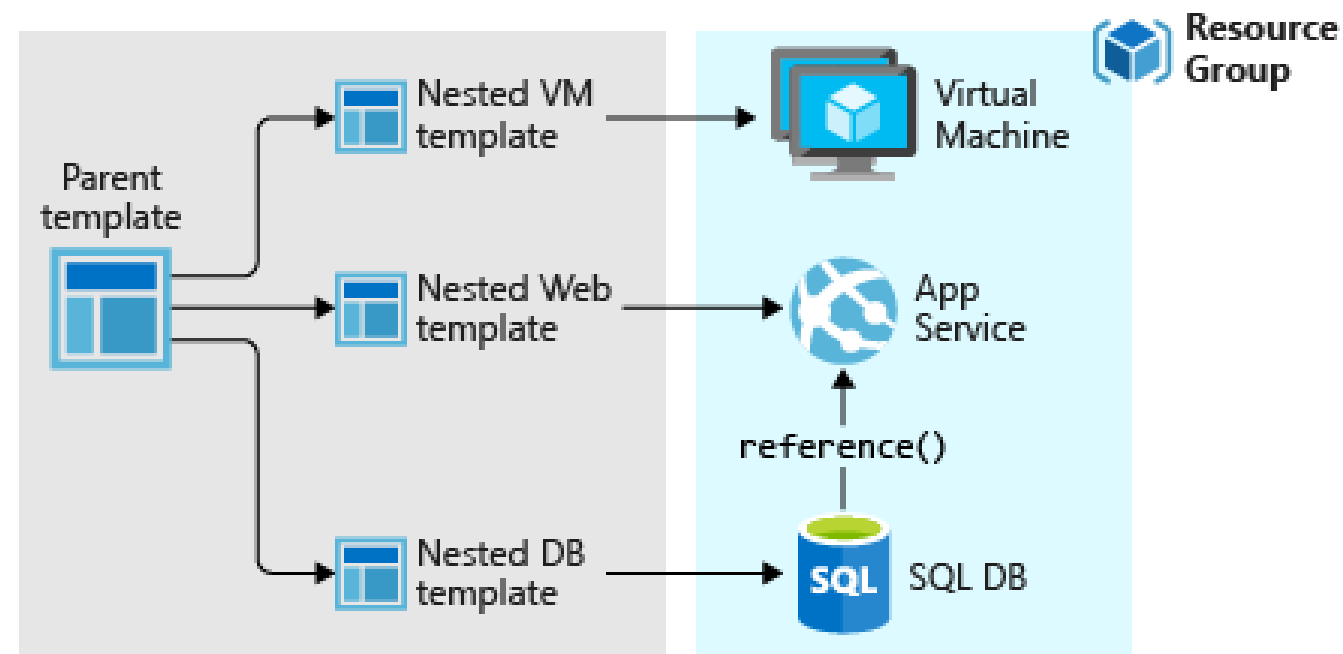
Users can create templates and resource groups depending on how they wish to manage the solution.



For the above example, one can deploy a three-tier application through a single template to a single resource group.

ARM Template: Design

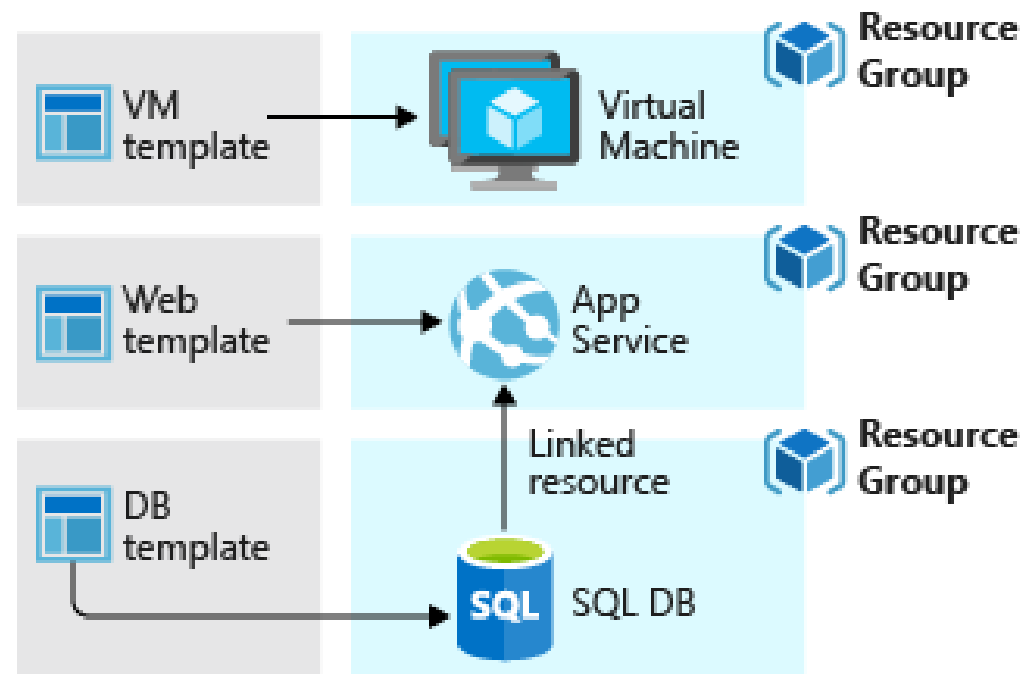
Users can create templates and resource groups depending on how they wish to manage the solution.



For the above example, one can deploy a three-tier solution through a parent template that includes three nested templates.

ARM Template: Design

Users can create templates and resource groups depending on how they wish to manage the solution.



For the above example, one can deploy three tiers to separate resource groups.



Template Schema

It defines all the resource manager resources in a deployment.

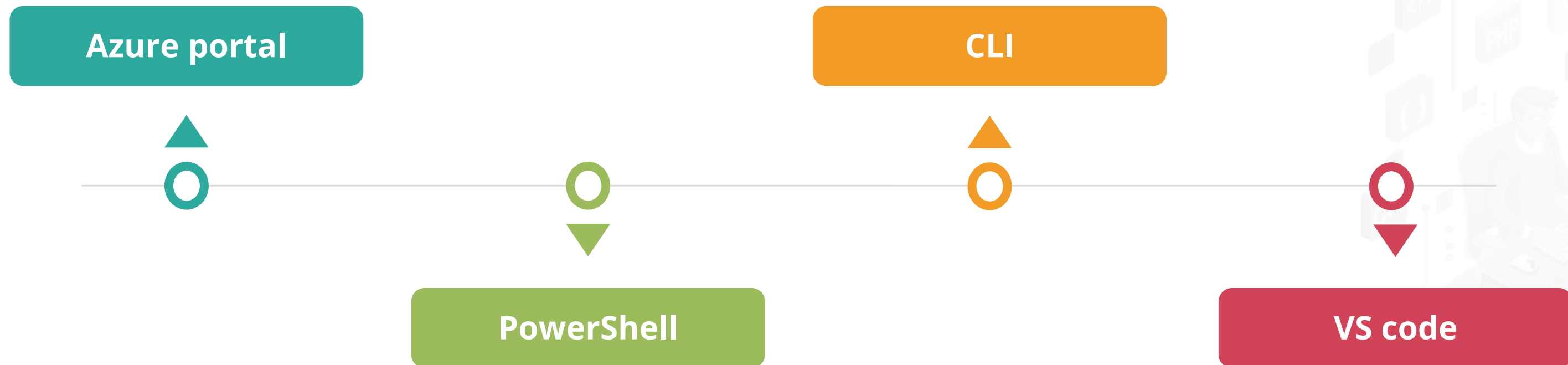
In its simplest structure, a template has the following elements:

```
{
  "$schema": "https://schema.management.azure.com/schemas/2019-04-01/deploymentTemplate.json#",
  "languageVersion": "",
  "contentVersion": "",
  "apiProfile": "",
  "definitions": { },
  "parameters": { },
  "variables": { },
  "functions": [ ],
  "resources": [ ], /* or "resources": { } with languageVersion 2.0
*/
  "outputs": { }
}
```



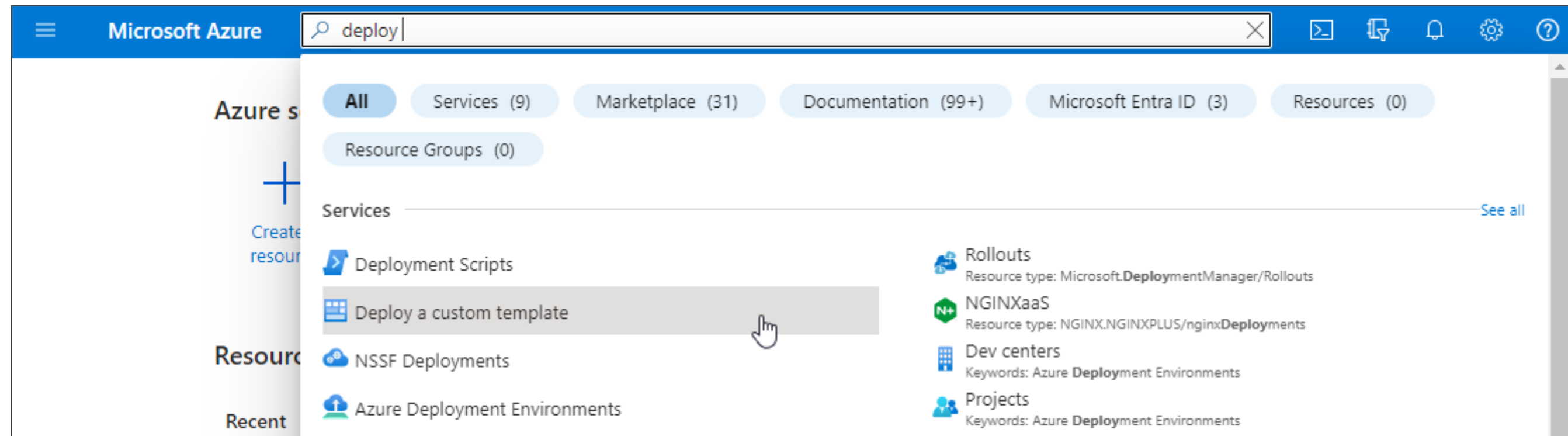
ARM Template Deployment Options

Users can deploy the templates by using:



ARM Template Deployment Options

Users can deploy the custom template from the **Azure portal**:



ARM Template Deployment Options

Users can deploy the template using **PowerShell**.

To deploy a local template, use the **-TemplateFile** parameter in the deployment command:

```
New-AzResourceGroupDeployment `
  -Name ExampleDeployment `
  -ResourceGroupName ExampleGroup `
  -TemplateFile <path-to-template>
```



ARM Template Deployment Options

Users can deploy the ARM template using the **CLI**.

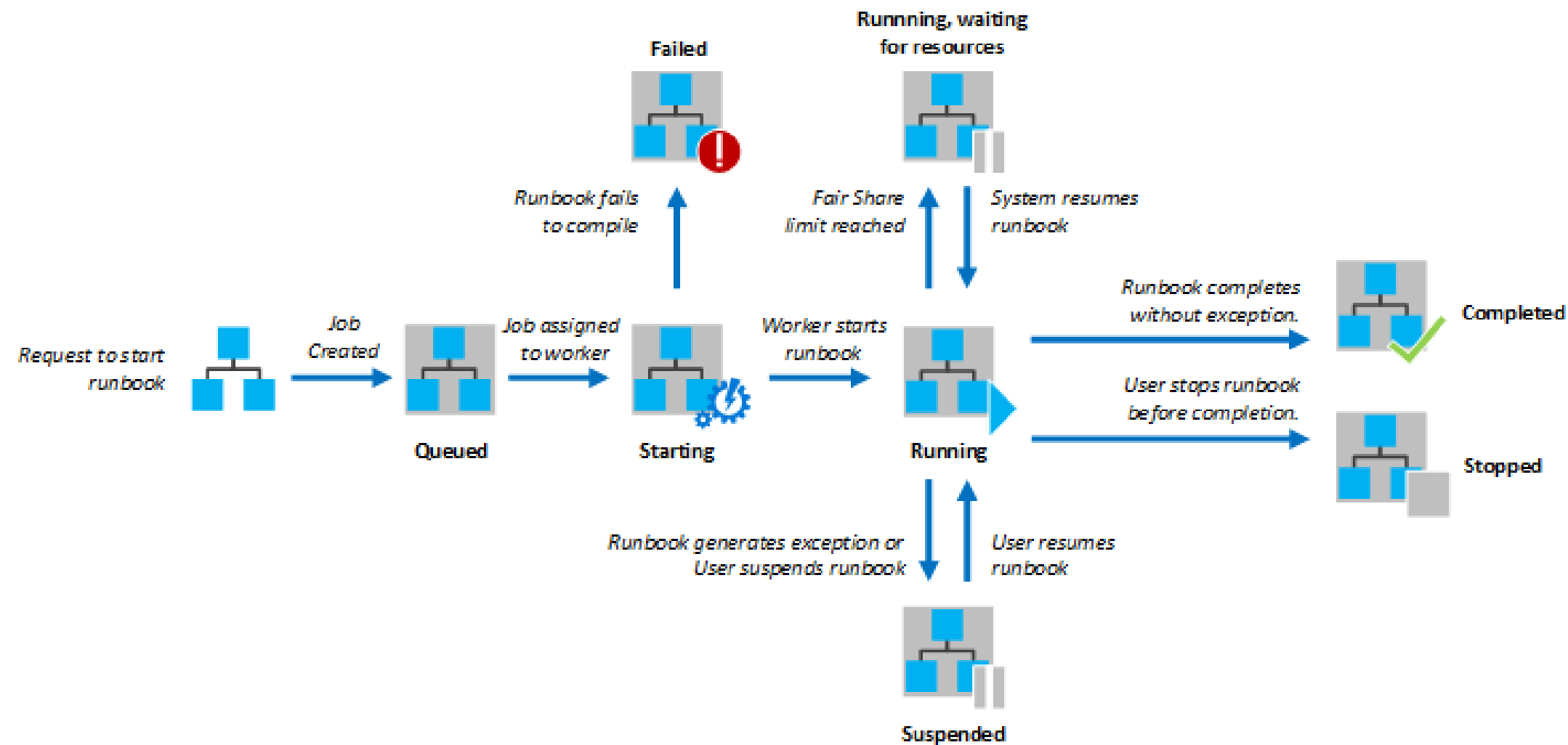
To deploy a local template, use the **--template-file** parameter:

```
az deployment group create
--name ExampleDeployment
--resource-group ExampleGroup
--template-file <path-to-template>
--parameters storageAccountType=Standard_GRS
```



Runbooks in Azure Automation

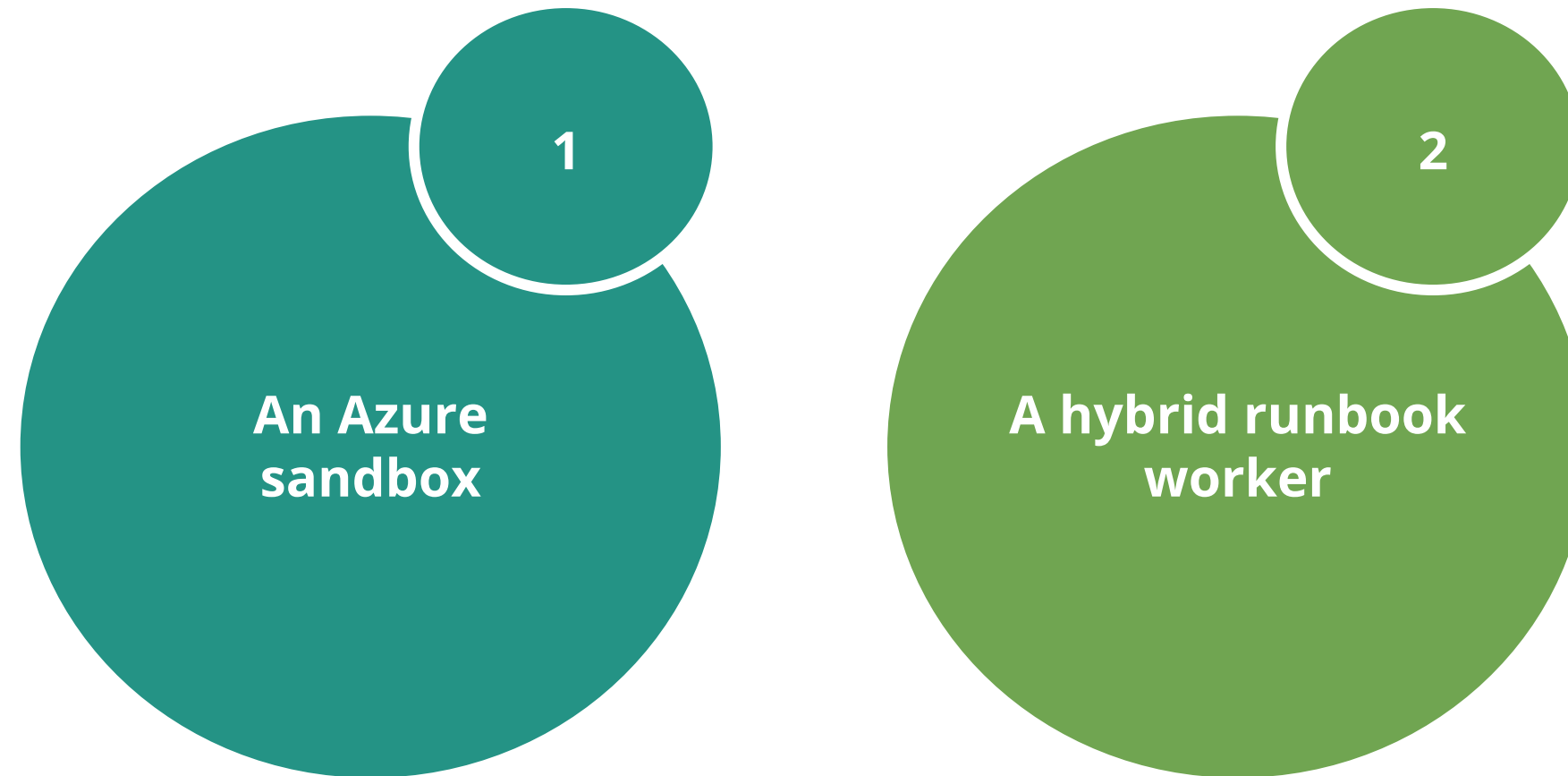
A runbook refers to a set of tasks or a workflow that is defined and executed in Azure Automation.



The above diagram shows the lifecycle of a runbook.

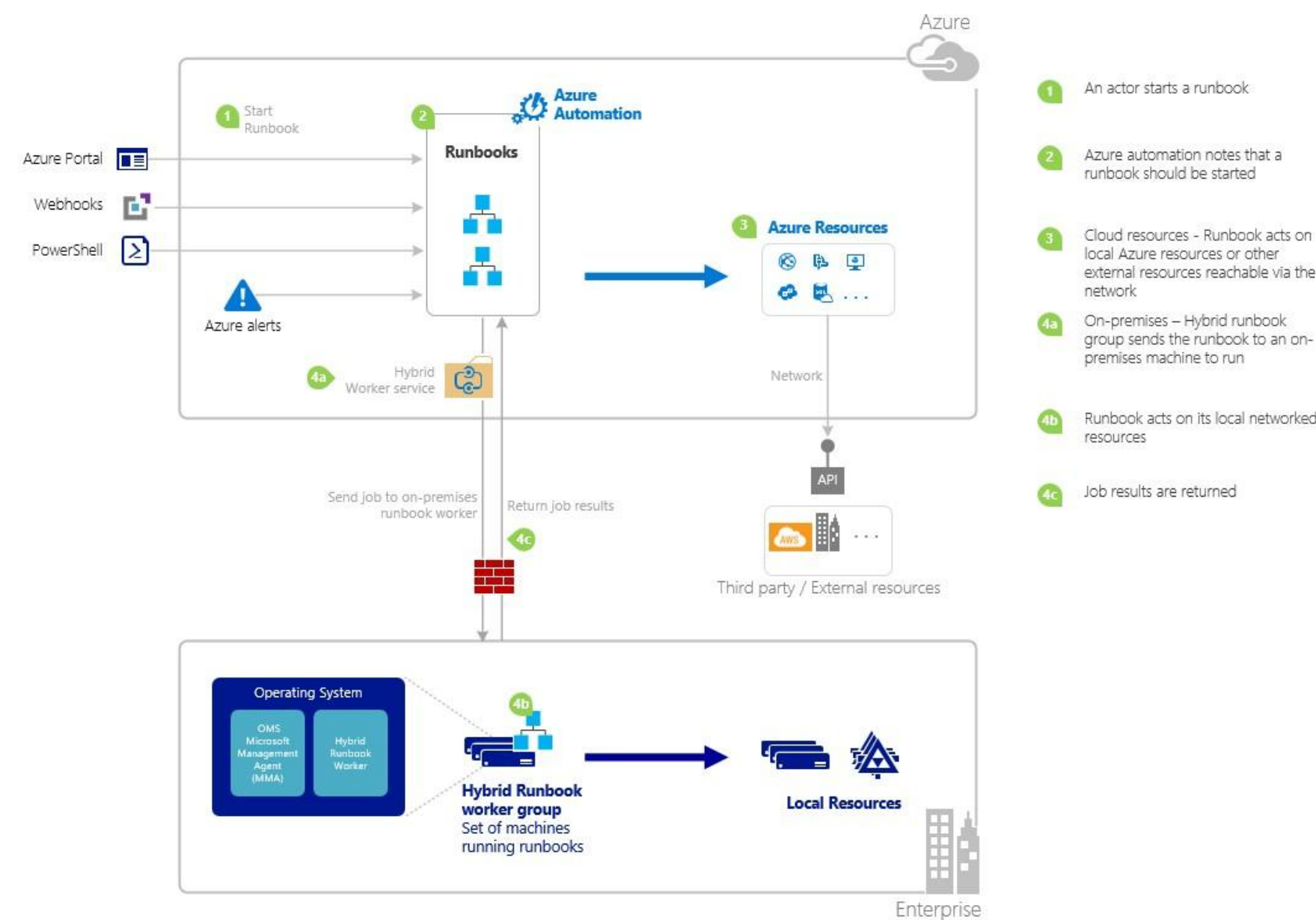
Runbooks in Azure Automation

Azure Automation runbooks can run in:



Runbooks in Azure Automation

The image demonstrates the complete runbook life cycle in Azure Automation, showcasing various methods for initiating a runbook:



Downloading ARM Template



Duration: 10 Min.

Problem Statement:

You have been given a task to download an ARM template for provisioning a Microsoft Azure virtual machine with the specified configurations and resources.

Outcome:

The ARM template is ready for use, enabling the seamless provisioning of a Microsoft Azure virtual machine with the specified configurations and resources, ensuring accuracy, compliance, and ease of deployment.

Note: Refer to the demo document for detailed steps: 02_Downloading_ARM_Template

ASSISTED PRACTICE

Assisted Practice: Guidelines

Steps to be followed:

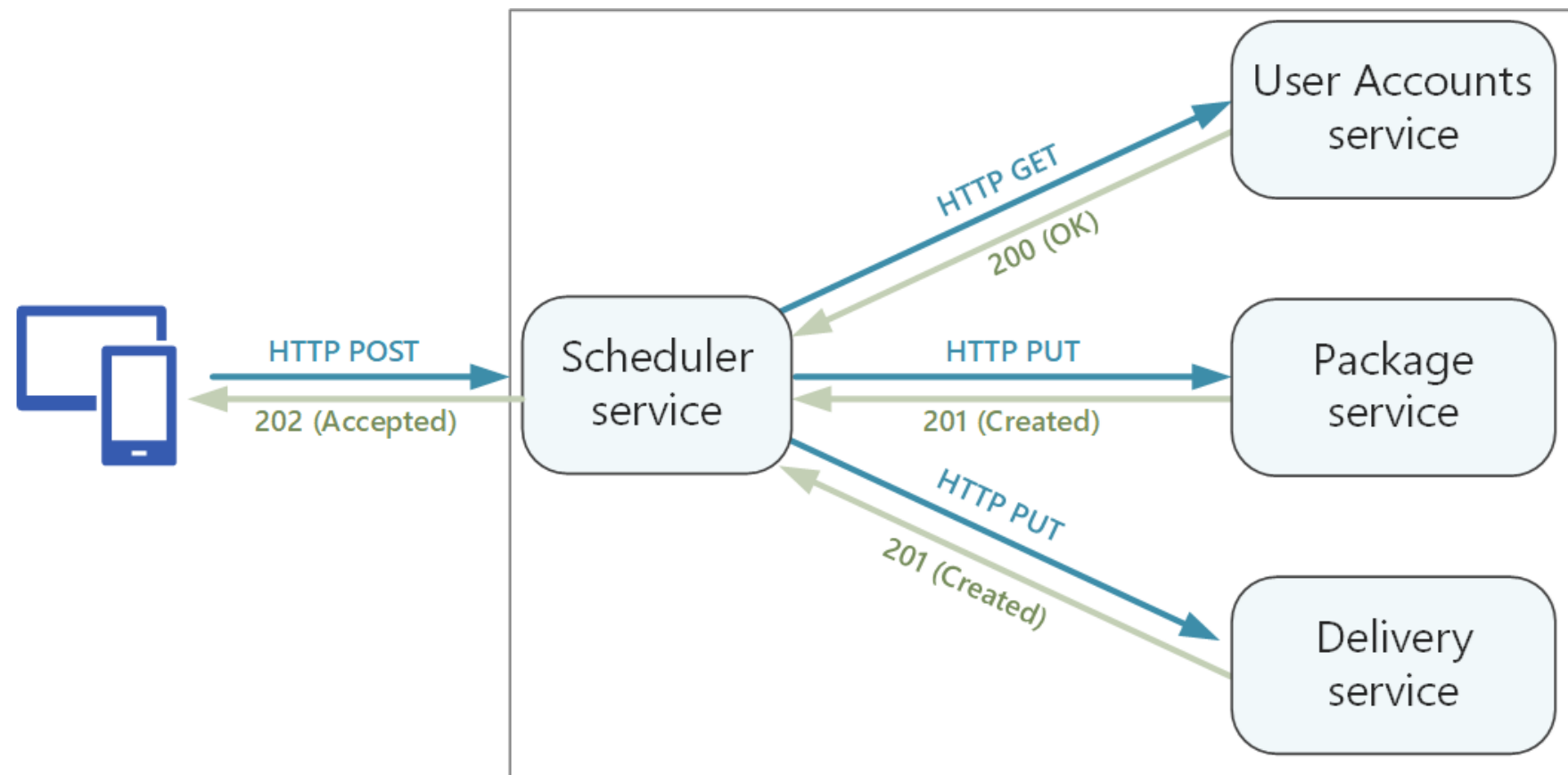
1. Download the ARM template



Recommend a Solution for API Integration

Designing APIs for Microservices

All data exchange between services happens either through messages or API calls.



APIs must have well-defined semantics and versioning schemes so that updates do not break other services.

Types of APIs

There are two types of APIs:



Public APIs

A public API must be compatible with client applications, typically browser applications or native mobile applications.



Backend APIs

Backend APIs facilitate interservice communication, which, depending on the granularity of services, may lead to network traffic.



Considerations of APIs

REST

It defines a uniform interface based on HTTP verbs. It includes well-defined semantics for idempotency, side effects, and response codes.

RPC

Based on operations or commands, RPC interfaces can affect the design with overly chatty APIs.

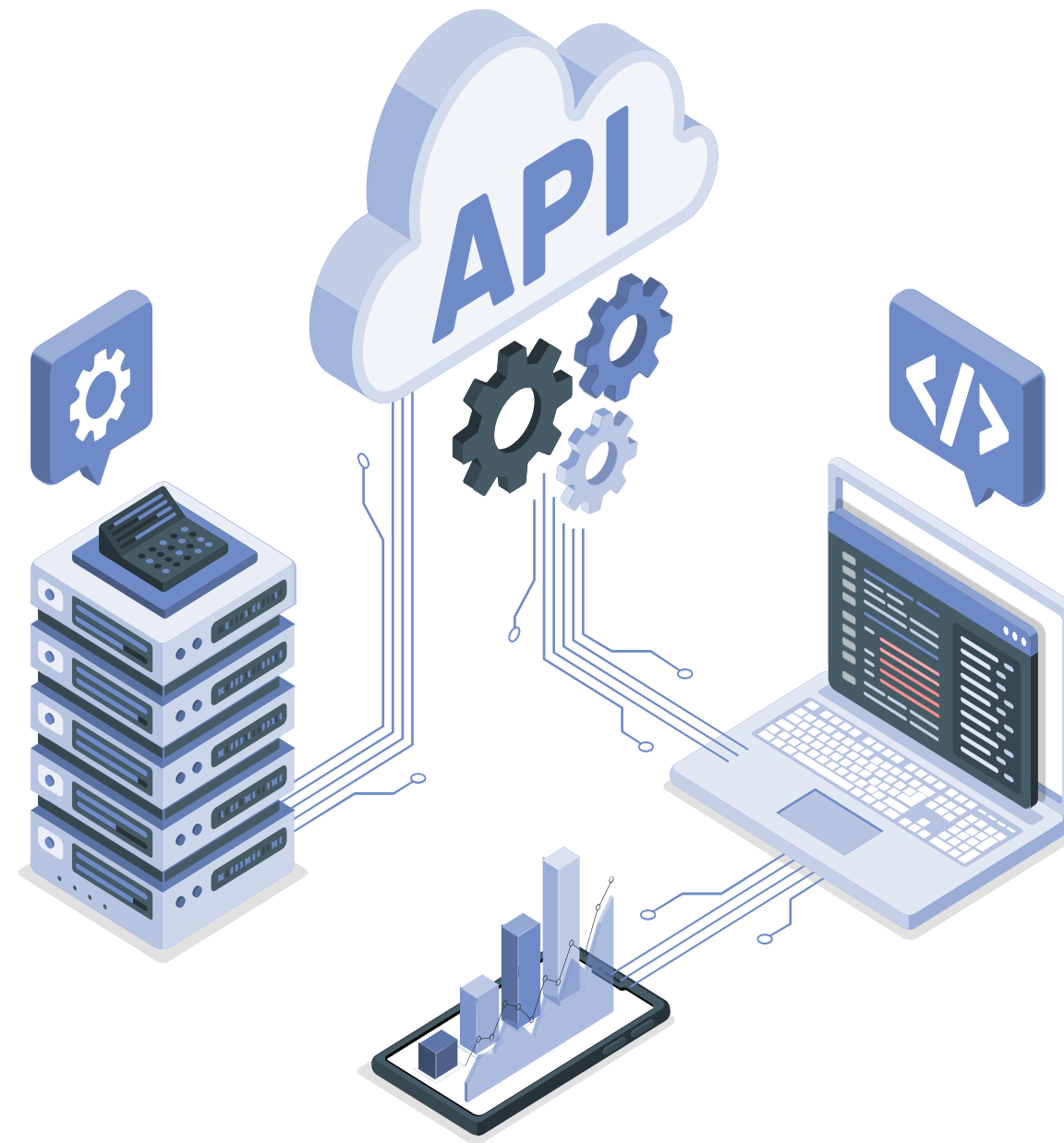
Baseline Recommendation

For a RESTful interface, the most common choice is REST over HTTP using JSON. For an RPC-style interface, there are several popular frameworks, including gRPC, Apache Avro, and Apache Thrift.



Azure API Management

It is a fully managed API gateway that enables organizations to create, publish, secure, and analyze APIs.



Azure API Management: Components

These components are Azure-hosted and fully managed by default:

API gateway

All requests from client applications first reach the API gateway, which then forwards them to respective backend services. The API gateway acts as a facade to the backend services.

Management plane

API providers interact with the service through the management plane, which provides full access to the API Management service capabilities.



Azure API Management: Components

These components are Azure-hosted and fully managed by default:

Developer portal

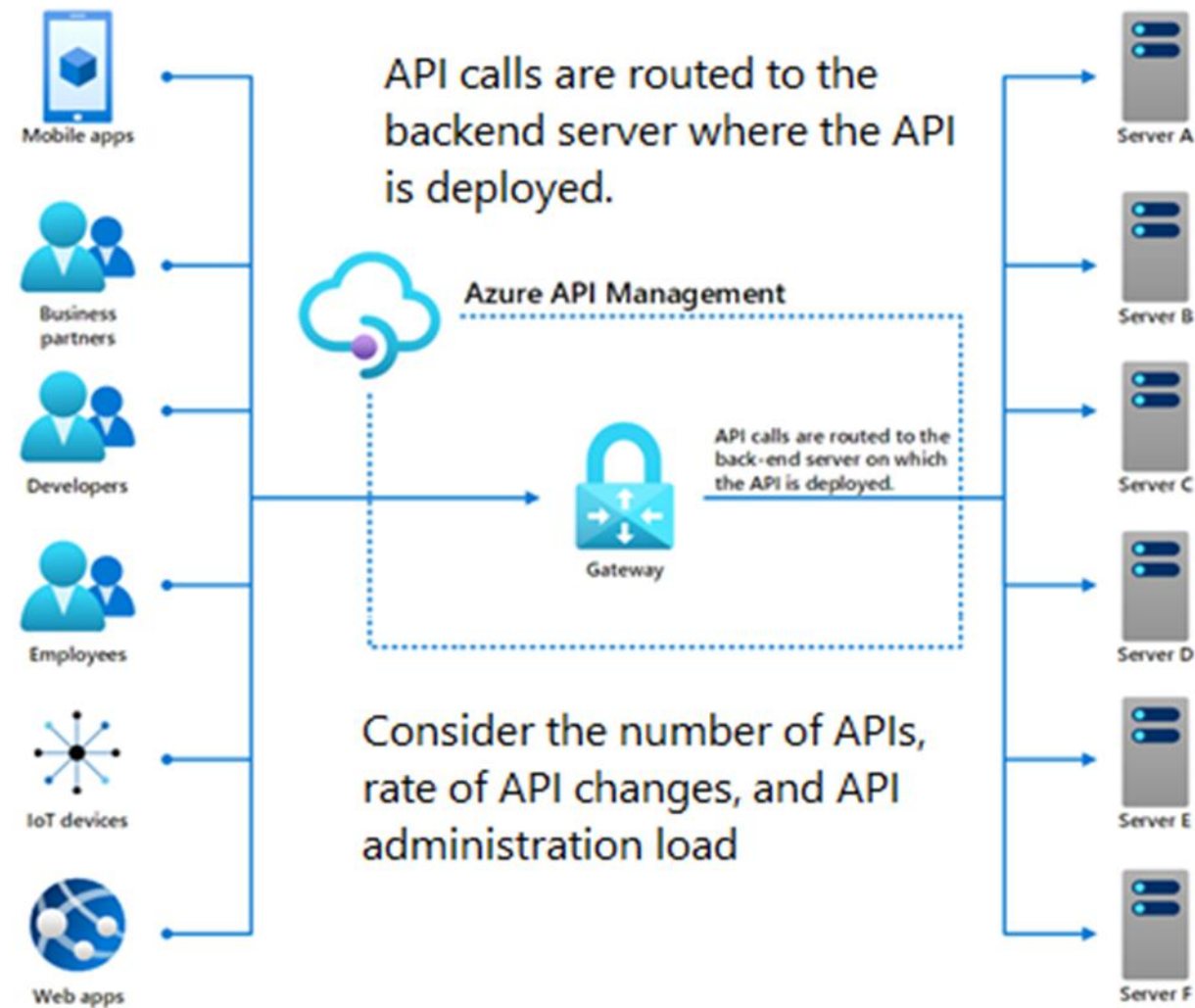
The open-source developer portal is an automatically generated, fully customizable website with the documentation of the user APIs.

Integration with Azure services

API Management integrates with many complementary Azure services to create enterprise solutions, including Azure Key Vault, Azure Monitor, Application Insights, and so on.



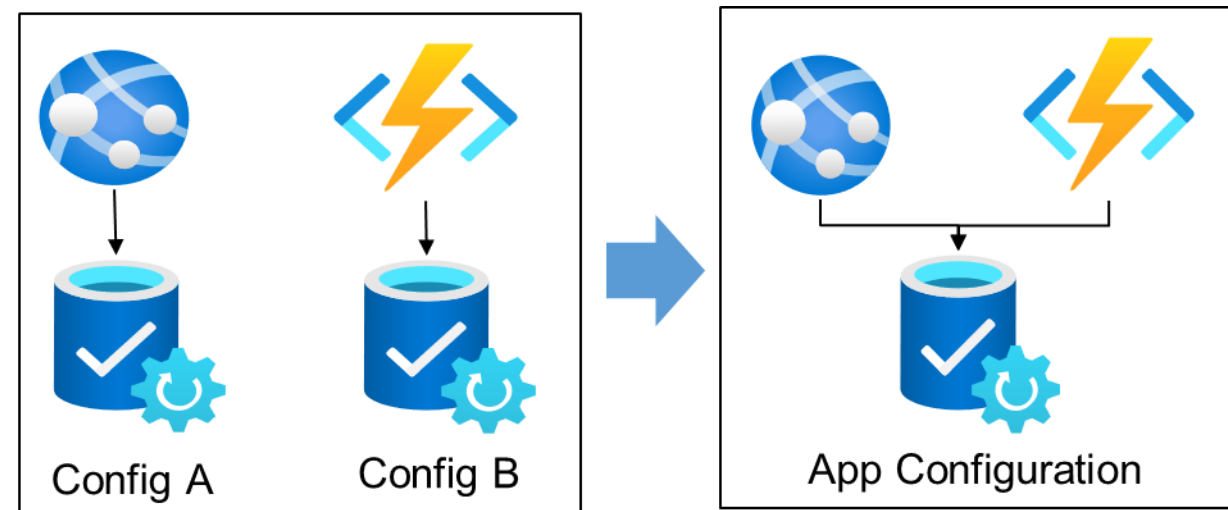
Designing an API Management Solution



It helps in:

- Centralizing the administration of multiple APIs under a unified umbrella
- Administering permissions and regulating access
- Enforcing compliance throughout the API landscape
- Standardizing API specifications
- Safeguarding APIs from unauthorized and malicious usage

Designing an App Configuration Solution



It helps in:

- Adapting key representations and mappings
- Replaying settings at specific points in time
- Providing a dedicated user interface for managing feature flags
- Comparing two sets of configurations based on custom-defined dimensions
- Heightening security with Azure-managed identities and encryption

TECHNOLOGY



Case Study

Design for Applications Architecture

Scenario

Simplilearn is looking to update its website to include customer-supplied product images in addition to the already existing photos provided by marketing. They believe that having more photos of products in use will give potential customers a better feel for how past customers loved their products after purchasing them. They do have some requirements as outlined below:

- **Uploaded images** will need to be scanned before getting posted on the website. Legal and Marketing are both requesting that after initial upload, the images be checked for any issues that reflect poorly upon the company or could cause legal issues. An in-house API has already been developed and deployed that can perform the necessary scanning.

Design for Applications Architecture

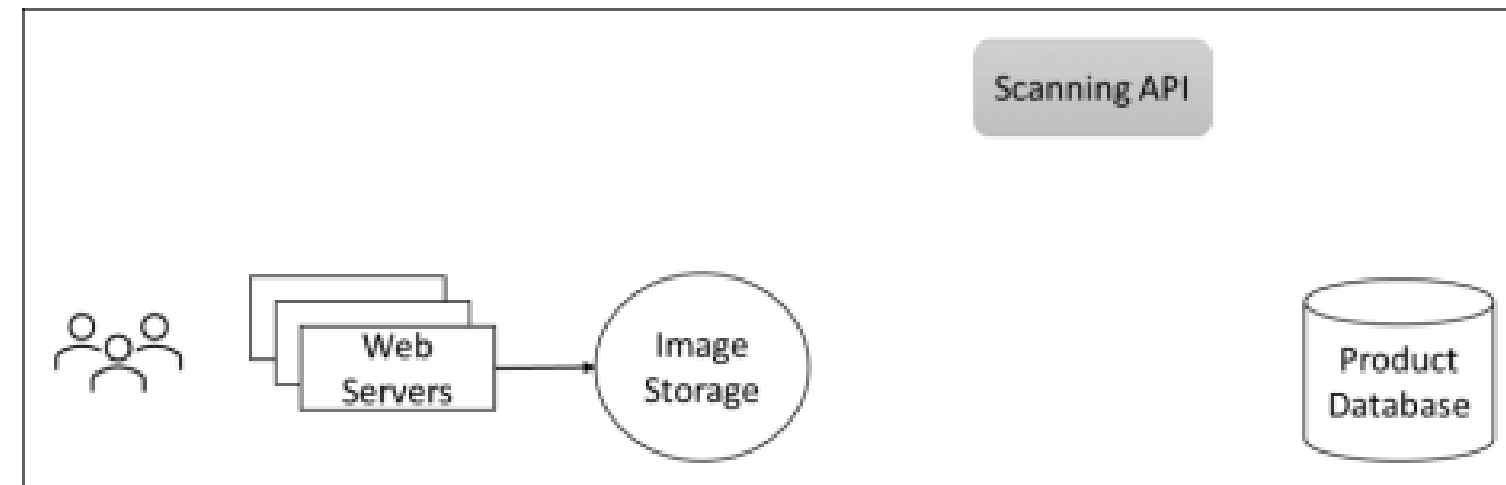
Scenario

- Based on **existing patterns**, Simplilearn expects the image uploads to happen very unevenly throughout the day. Certain periods may experience more uploads than the scanning software can handle, while other periods may experience very few or no uploads.
- Once an uploaded image has been scanned and approved by the system, “Simplilearn” would like for the customer to be sent an email thanking them for sharing their image.

Design for Applications Architecture

Scenario

- Cost and management of the solution is a concern, especially since “Simplilearn” isn’t sure how popular this feature will be initially. Minimize costs and leverage serverless solutions where possible.



Design for Applications Architecture

Requirements

Design an architecture for the customer images to be added to the company website.

- Where should the images be stored?
- How will you ensure that all images are scanned even when uploads are outpacing scanning?
- Once images are approved and the catalog database is updated, how will the customer be notified?

How are you incorporating the Well Architected Framework pillars to produce a high-quality, stable, and efficient cloud architecture?

Key Takeaways

- Azure Functions allows the user to write less code, maintain less infrastructure, and save on costs.
- Azure Kubernetes Service simplifies deploying a managed Kubernetes cluster in Azure by offloading the operational overhead to Azure.
- Azure Automation runbooks can run in an Azure sandbox and a hybrid runbook worker.
- REST defines a uniform interface based on HTTP verbs. It includes well-defined semantics for idempotency, side effects, and response codes.





Knowledge Check

**Knowledge
Check**

01

Which among the following allows users to run small pieces of code without worrying about application infrastructure?

- A. Azure Kubernetes Service
- B. Azure Logic Apps
- C. Azure Functions
- D. Azure Monitor



Knowledge
Check

01

Which among the following allows users to run small pieces of code without worrying about application infrastructure?

- A. Azure Kubernetes Service
- B. Azure Logic Apps
- C. Azure Functions
- D. Azure Monitor



The correct answer is **C**

Azure Functions allow users to run small pieces of code without worrying about application infrastructure.

**Knowledge
Check**
02

Which of the following services is used to manage resources in Azure?

- A. Log Analytics
- B. Azure Resource Manager
- C. Azure portal
- D. None of these



Knowledge
Check
02

Which of the following services is used to manage resources in Azure?

- A. Log Analytics
- B. Azure Resource Manager
- C. Azure portal
- D. None of these



The correct answer is **B**

Azure Resource Manager manages resources in Azure.

**Knowledge
Check**
03

Which among the following can create and run automated workflows with little to no code?

- A. Azure Resource Manager
- B. Azure Monitor
- C. Azure Logic Apps
- D. Azure Kubernetes Service



**Knowledge
Check
03**

Which among the following can create and run automated workflows with little to no code?

- A. Azure Resource Manager
- B. Azure Monitor
- C. Azure Logic Apps
- D. Azure Kubernetes Service



The correct answer is **C**

Azure Logic Apps is a cloud platform where one can create and run automated workflows with little to no code.

**Knowledge
Check**
04

Which type of API requires no special libraries?

- A. HTTP
- B. REST
- C. RPC
- D. None of these



Knowledge
Check

04

Which type of API requires no special libraries?

- A. HTTP
- B. REST
- C. RPC
- D. None of these



The correct answer is **B**

REST API requires no special libraries.

Design Solution for App Architecture

Duration: 25 min.



Project Agenda: To design the Azure Kubernetes Service (AKS) cluster for the given scenario

Description: You work as an architect for an organization building a hospitality management application. The application will consist of microservices hosted on Azure. Your task is to decide on the appropriate services for this application. Additionally, you need to provide developers with the ability to proactively identify and fix performance issues. They should also be able to demonstrate user connections to the room booking service.

Steps to be followed:

1. Create an Azure Kubernetes Service (AKS) cluster

TECHNOLOGY

Thank You